

Learning Material: CMU 11-785

Introduction to Deep Learning

Lecture 2-3

Presented by Yalei Zhao

Outline

Neural network structure

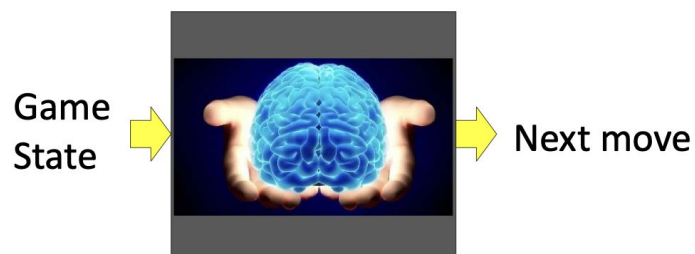
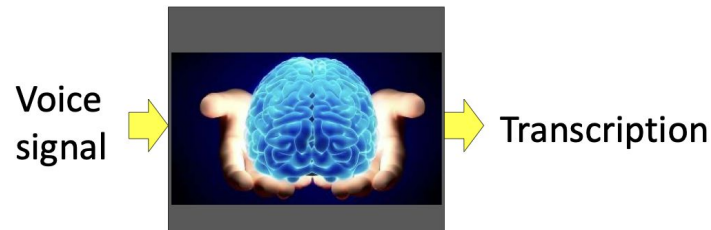
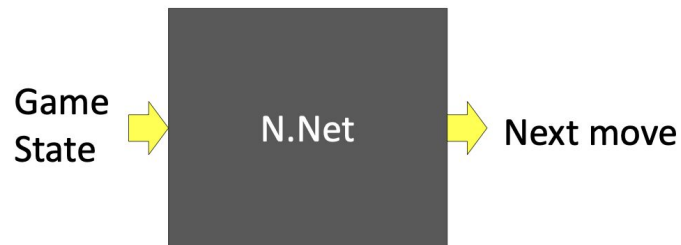
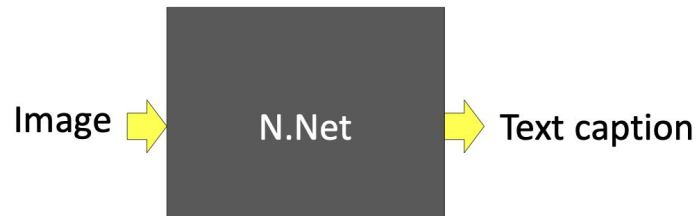
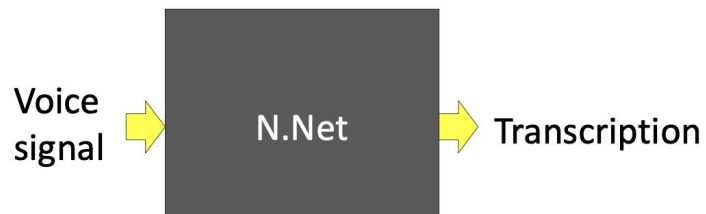
Examples: Single-layer to deep NN

Activation Functions

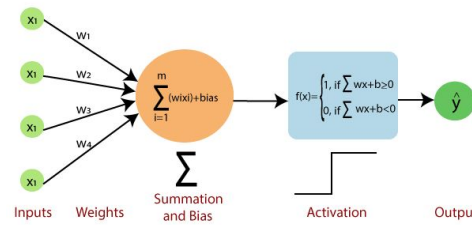
Loss and empirical loss

What are neuronal networks?

Functions that take an input and give an output



Neural network structure



neuron:

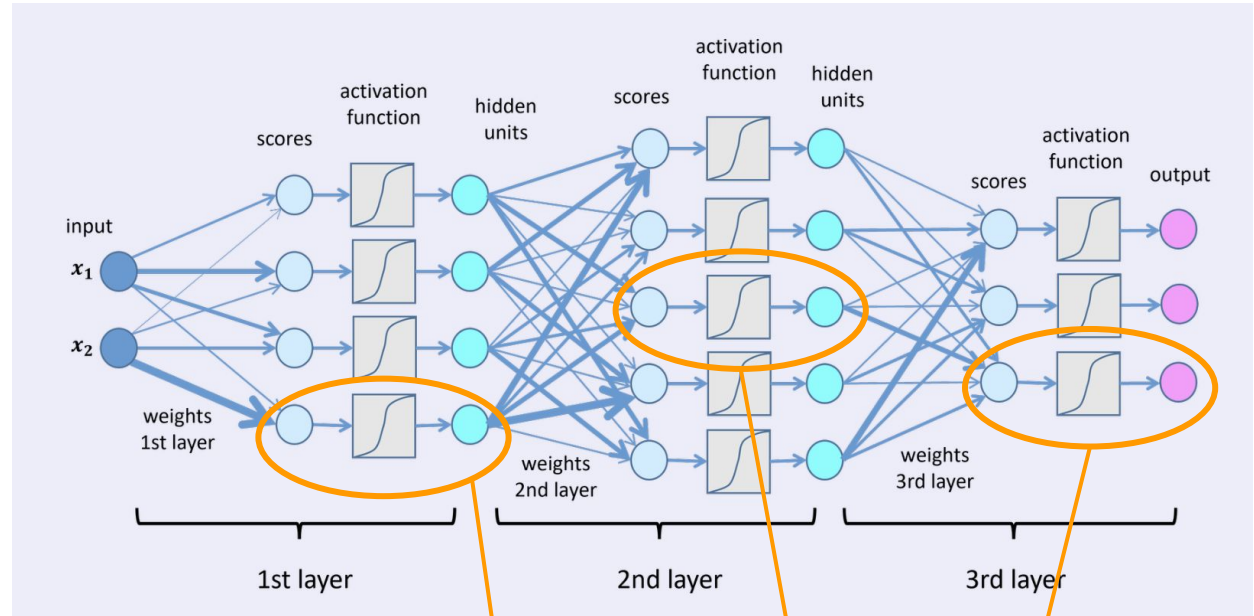
A basic computational unit that computes a weighted sum of inputs and applies a nonlinear activation

layers:

= number of weight layers =
number of neuron layers (exclude input, include output)

activation function:

A math function applied to a neuron's pre-activation to produce its output (e.g., ReLU, sigmoid, softmax).

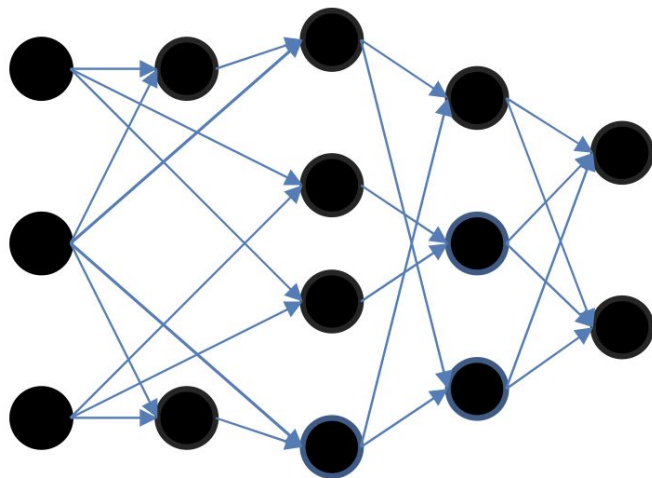


Sometimes drawn as one neuron in some neuron network illustrations

Layers = number of weight layers

= number of neuron layers (exclude input, include output)

A layer is a set of neurons that are all at the same depth with respect to the input
(consider the longest path from source to this neuron)



Input:

Layer 1:

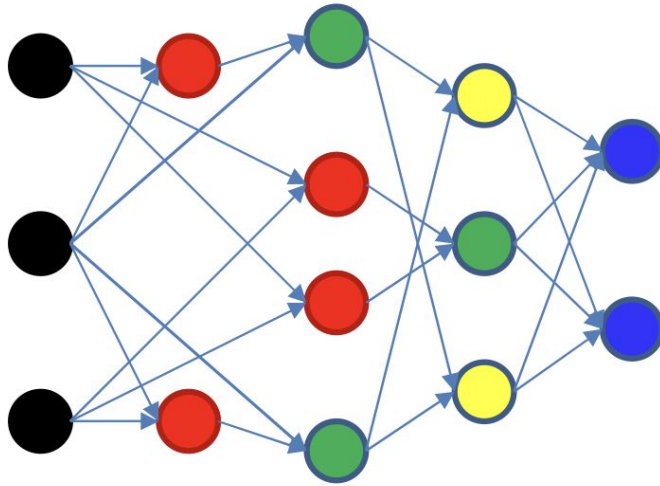
Layer 2:

Layer 3:

Layer 4:

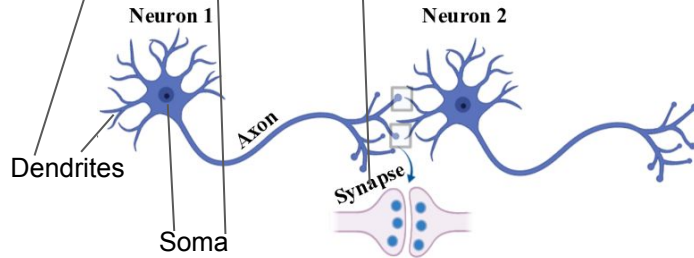
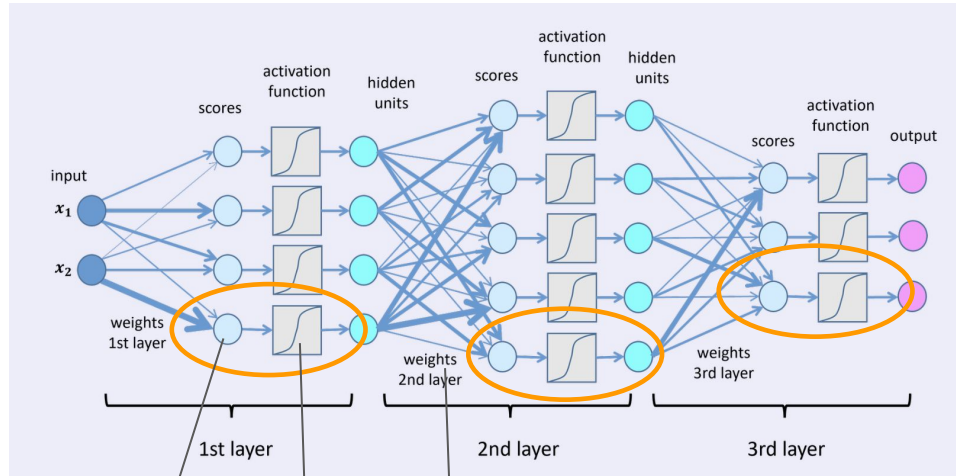
Layers

A layer is a set of neurons that are all at the same depth with respect to the input (consider the longest path from source to this neuron)



Input: Black
Layer 1: Red
Layer 2: Green
Layer 3: Yellow
Layer 4: Blue

Neural network neurons compared with human neurons



NN neurons

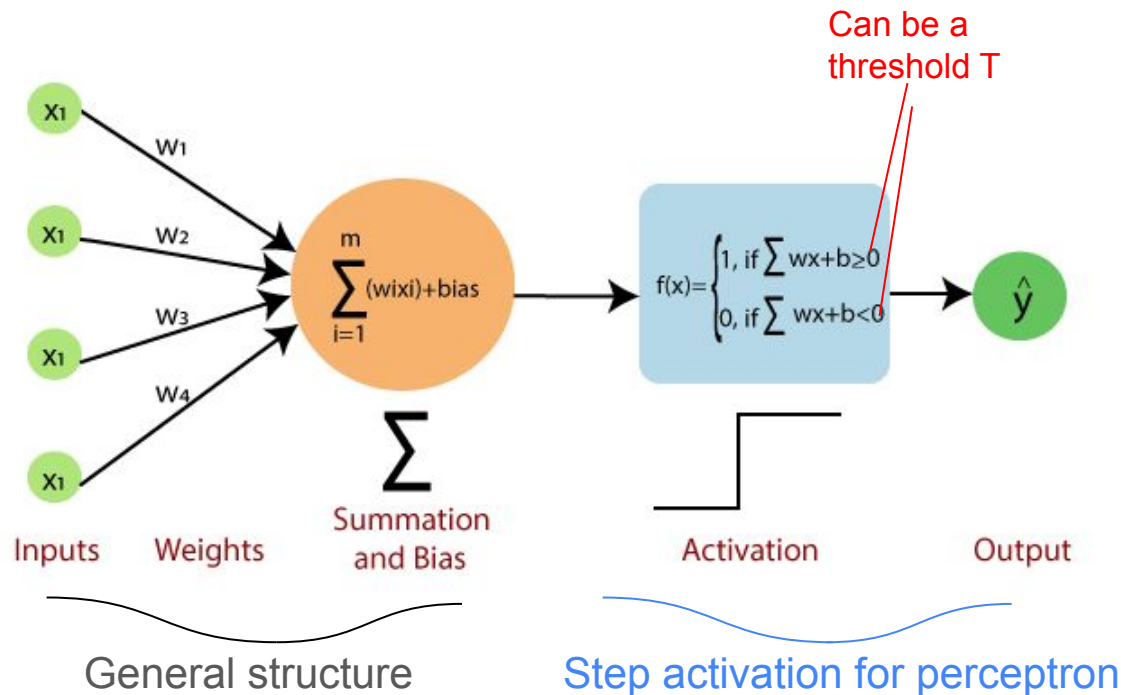
1. **Inputs x** come from previous layer
2. **Weights w** scale the inputs
3. **Weighted sum + activation** compute output
4. **Output y** goes through weight edges to next layer

Human neurons

1. Dendrites receive **signals from other neurons**
2. Synapses modulate **signal strength**
3. **Soma**: integrate signals
4. **Axon transport signals** to next neurons

Single-layer neural network

Example: one-layer perceptron



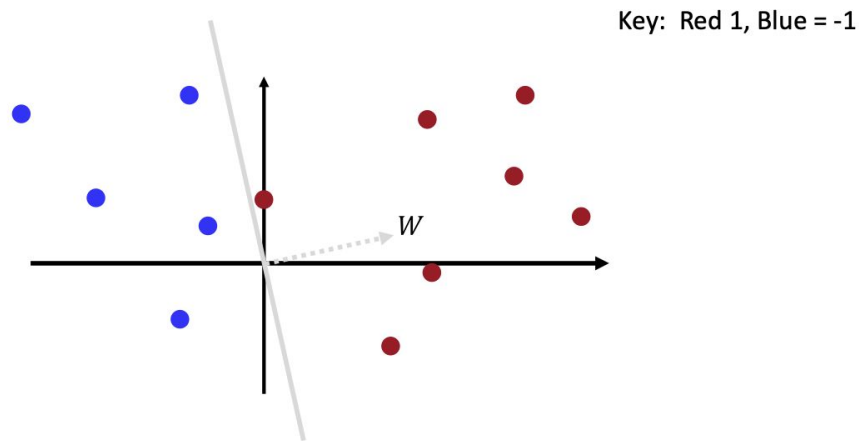
Bias (intercept) can be introduced explicitly (+bias) or setting: $x_3=1, w_3=\text{bias}$

For perceptron:

- Activation function is a step function, with threshold T
- Output is binary 0 or 1

Single-layer NN - Perceptron Learning Algorithm

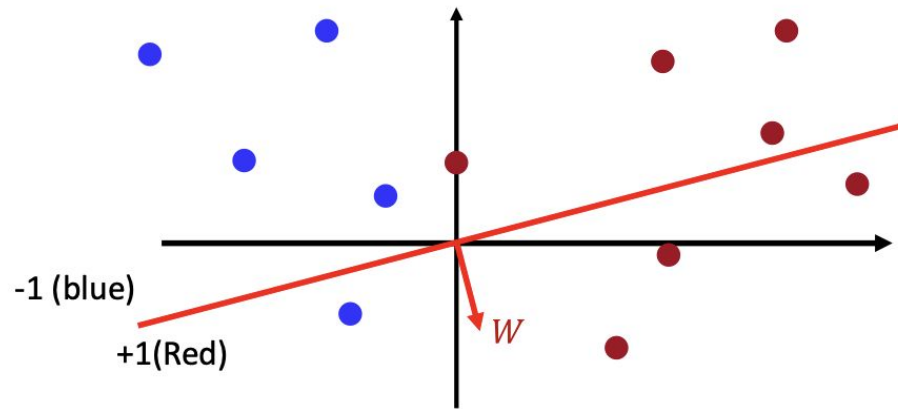
The Perceptron Problem



- Learning the perceptron: Find the weights vector W such that the plane described by $W^T X = 0$ perfectly separates the classes
 - $W^T X$ is positive ($W^T X > 0$) for all red dots
 - The angle between W and positive-class vectors is less than 90
 - $W^T X$ is negative ($W^T X < 0$) for all blue dots
 - The angle between W and negative-class vectors is greater than 90

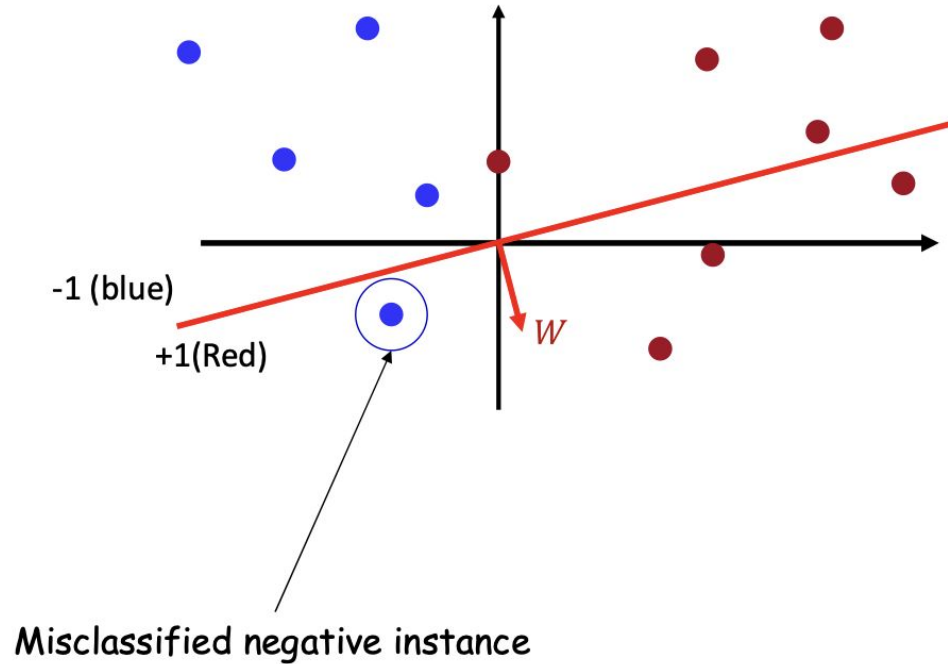
When
perfectly
separated:

A Simple Method: The Perceptron Algorithm

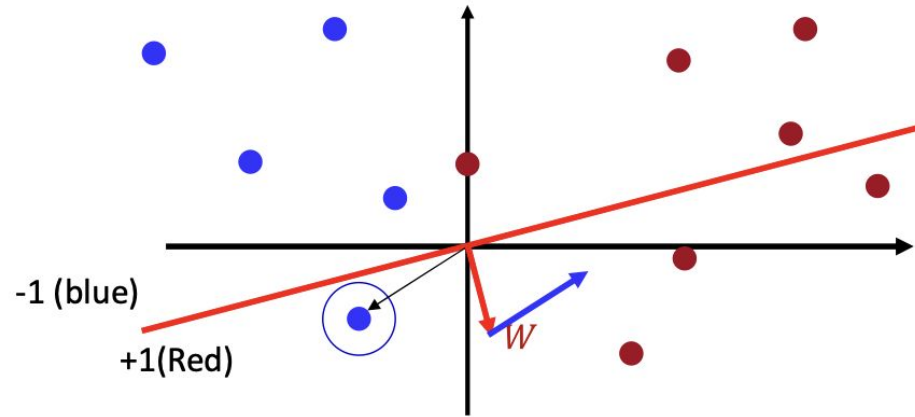


- **Initialize:** Randomly initialize the hyperplane
 - I.e. randomly initialize the normal vector W
- **Classification rule** $\text{sign}(W^T X)$
 - Vectors on the same side of the hyperplane as W will be assigned +1 class, and those on the other side will be assigned -1
- The random initial plane will make mistakes

Perceptron Algorithm

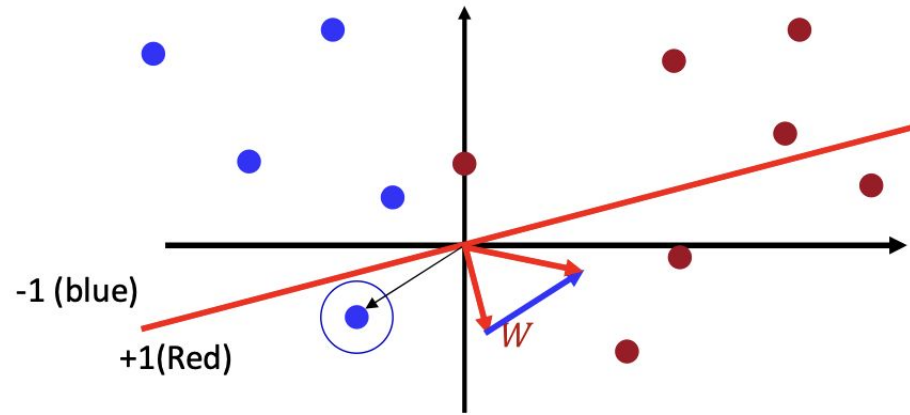


Perceptron Algorithm



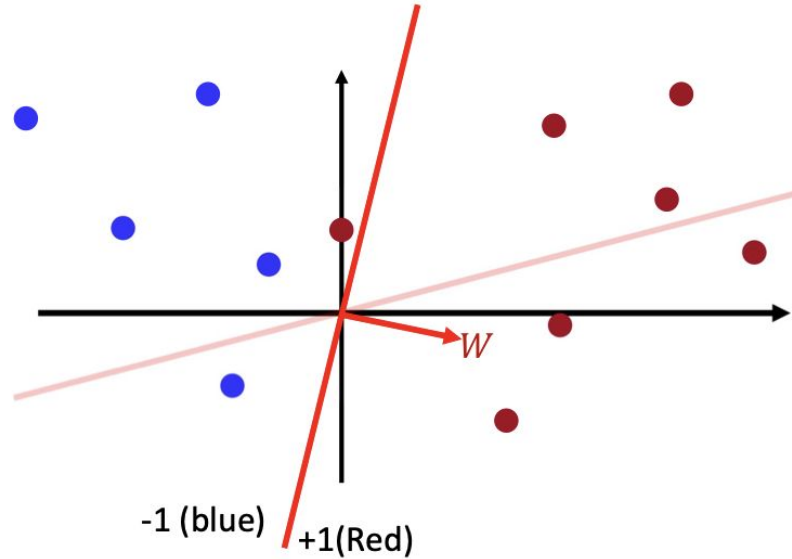
Misclassified *negative* instance, *subtract* it from W

Perceptron Algorithm



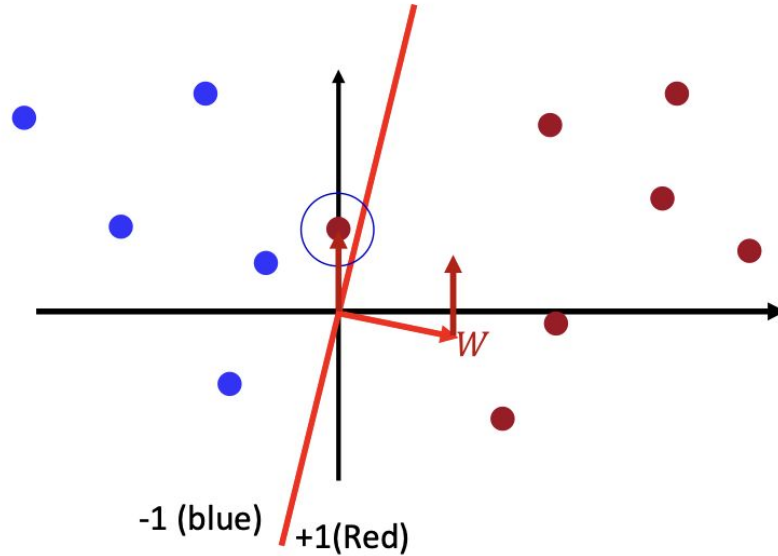
The new weight

Perceptron Algorithm



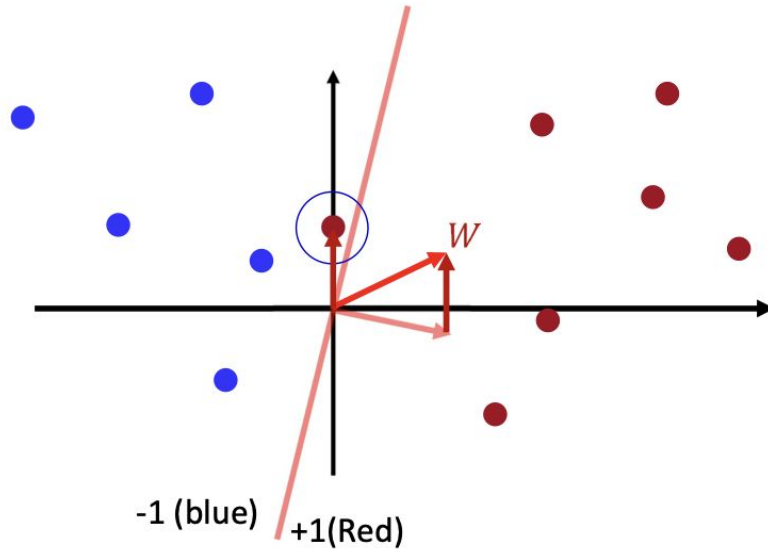
The new weight (and boundary)

Perceptron Algorithm



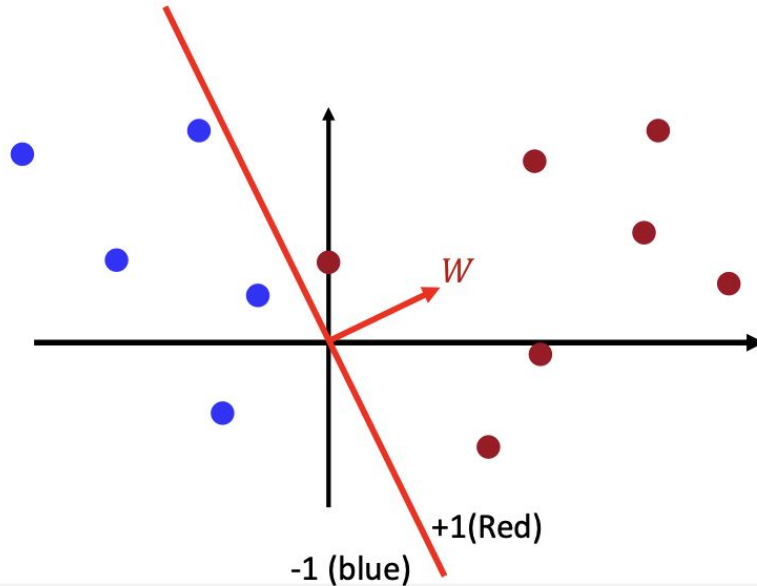
Misclassified *positive* instance, *add* it to W

Perceptron Algorithm



The new weight vector

Perceptron Algorithm



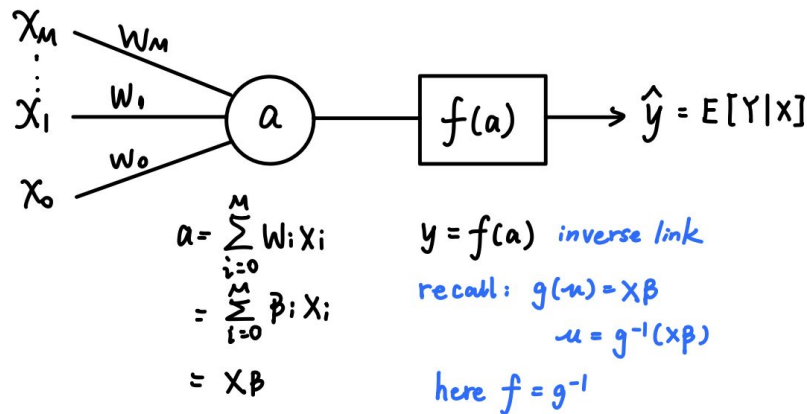
The new decision boundary

Perfect classification, no more updates, we are done

If the classes are linearly separable, guaranteed to converge in a finite number of steps

Single-layer neural network

Example: Linear regression, GLMs

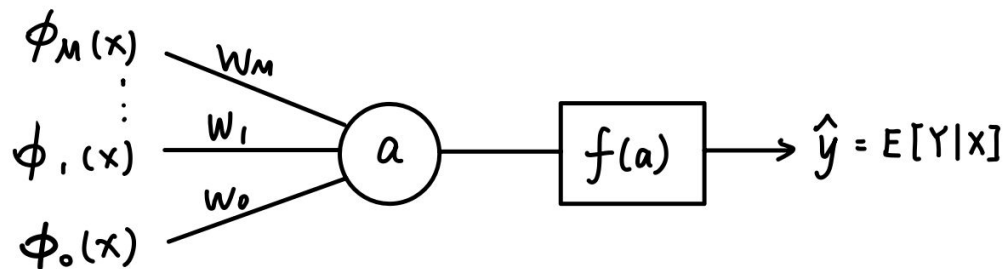


- Linear regression : $E[Y|X] = f(a) = a = X\beta$
- Logistic regression : $E[Y|X] = f(a) = \frac{1}{1 + e^{-a}} = \frac{1}{1 + e^{-X\beta}}$
- Poisson regression : $E[Y|X] = f(a) = e^a = e^{X\beta}$

Single-layer neural network

Example: GLM with basis functions

Basis functions



e.g. polynomial basis $\phi_j(x) = x^j$

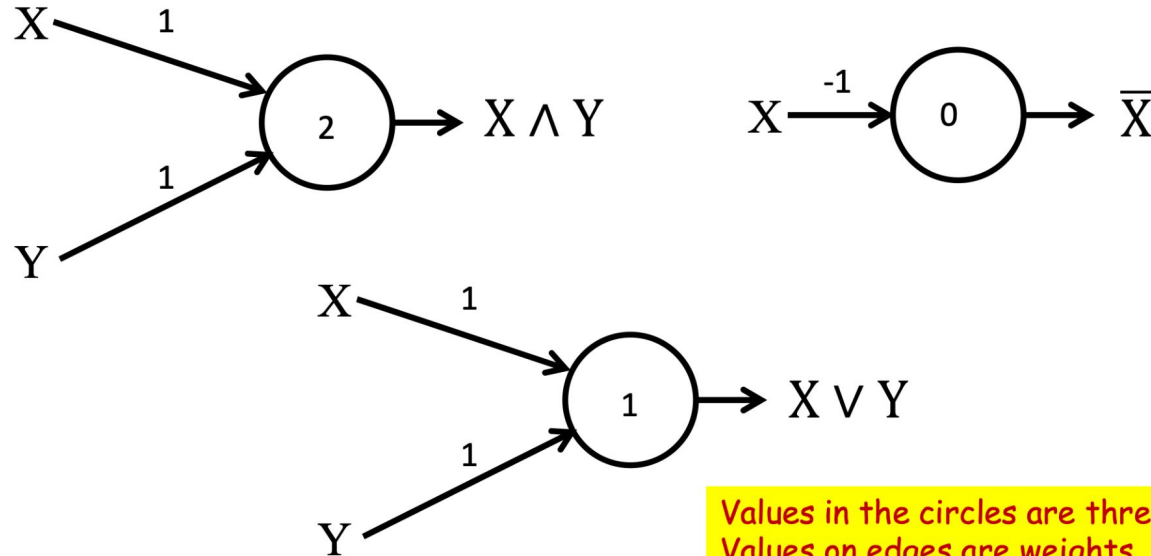
Transformation in x

Gaussian basis $\phi_j(x) = \exp \left[-\frac{(x - \mu_j)^2}{2\sigma^2} \right]$

Single-layer neural network

Example: AND OR NOT gate (inputs 0 or 1)

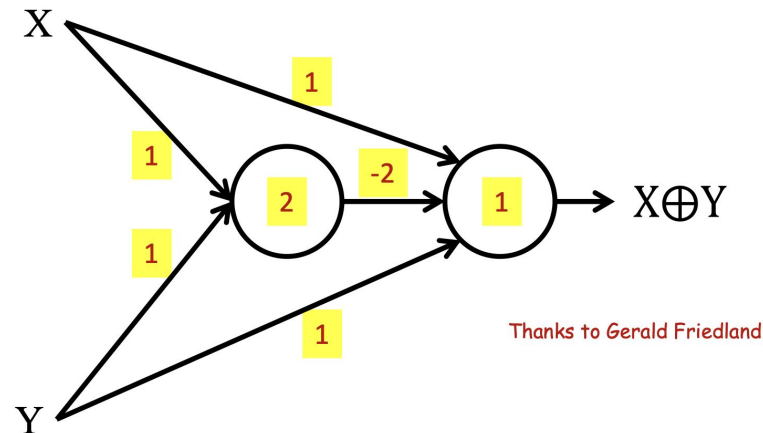
$X=0, -X=0 \geq 0, \text{ out} = 1$
 $X=1, -X=-1 < 0, \text{ out} = 0$



Values in the circles are thresholds
Values on edges are weights

Double-layer neural network

Example: XOR gate (inputs 0 or 1)



Thanks to Gerald Friedland

$$X - 2 * I(X+Y \geq 2) + Y = X - 2 * (X \wedge Y) + Y$$

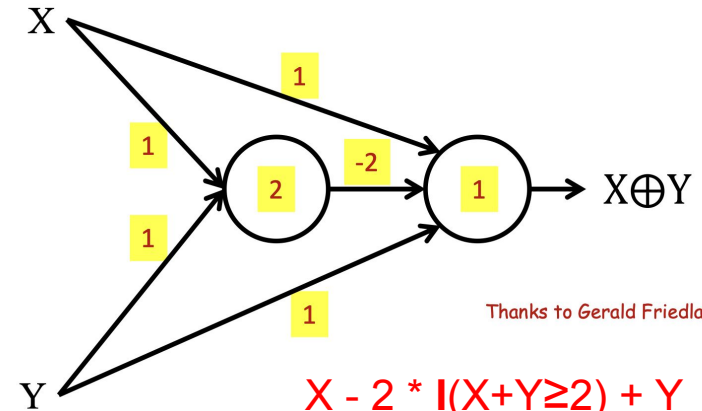
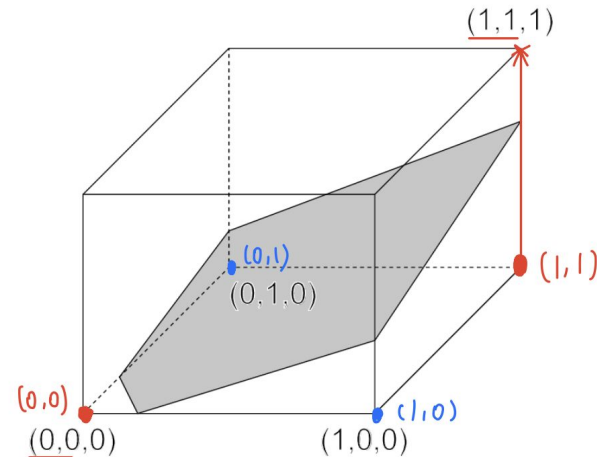
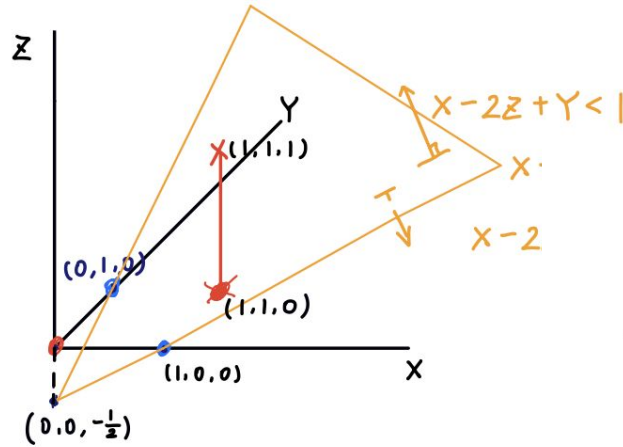
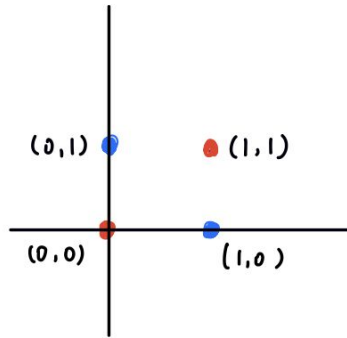
X	Y	X (XOR) Y
0	0	0
1	1	0
0	1	1
1	0	1

X	Y	$X+Y \geq 2$	$Z = X \wedge Y$	$X - 2Z + Y$
0	0	0	0	0
1	1	1	1	0
0	1	0	0	1
1	0	0	0	1

Equivalent

Double-layer neural network

Example: XOR gate (inputs 0 or 1)



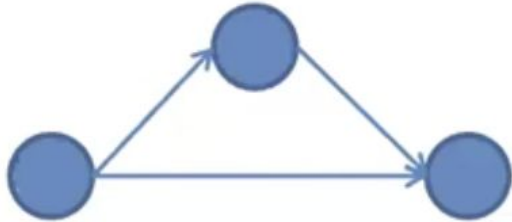
Thanks to Gerald Friedland

$$\begin{aligned} & X - 2 * I(X+Y \geq 2) + Y \\ &= X - 2 * (X \wedge Y) + Y \\ &= X - 2 * Z + Y \end{aligned}$$

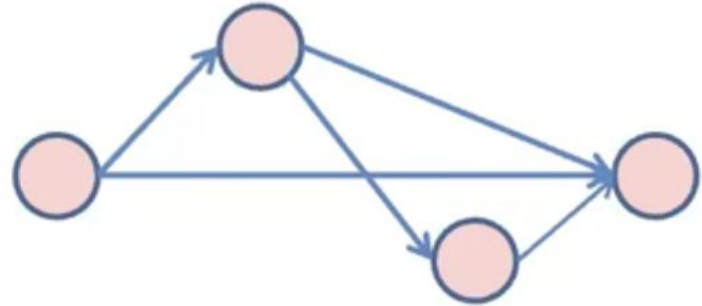
Deep neural network

Depth: Longest path from source to sink = number of weights layers

Deep: depth ≥ 3 is considered deep



Depth = 2



Depth = 3

Deep

Deep neural network

Example: Multiple Layer Perceptron (MLP)

- Inputs are real or Boolean stimuli
- Outputs are real or Boolean values
 - Can have multiple outputs for a single input

$$((A \& \bar{X} \& Z) | (A \& \bar{Y})) \& ((X \& Y) | (\bar{X} \& \bar{Z}))$$

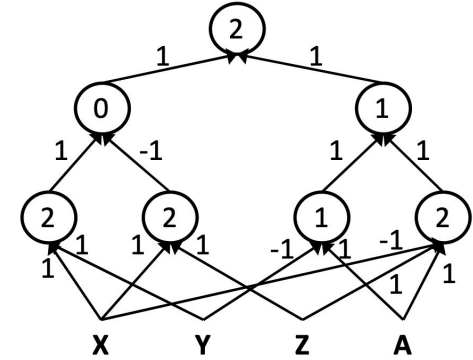


Figure 1

- Multi-layer Perceptrons as universal Boolean functions (Figure 1)
- MLPs as universal classifiers (Figure 2)
- MLPs as universal approximators (next page)

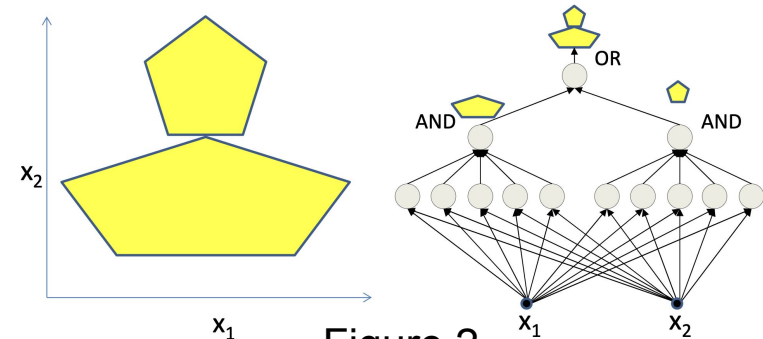


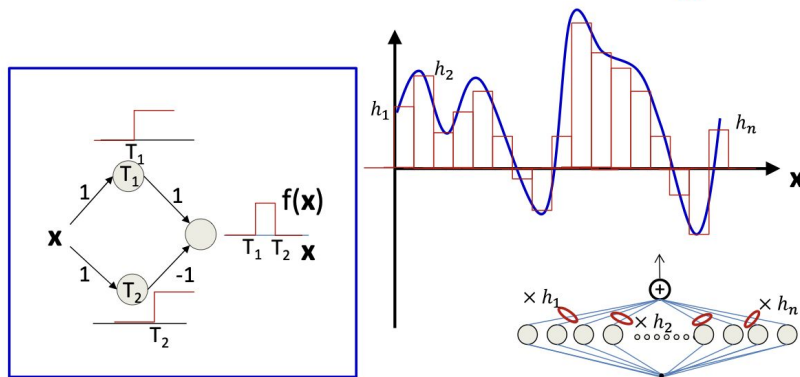
Figure 2

Deep neural network

Example: Multiple Layer Perceptron (MLP)

- Multi-layer Perceptrons as universal Boolean functions
- MLPs as universal classifiers
- MLPs as universal approximators (for continuous functions)

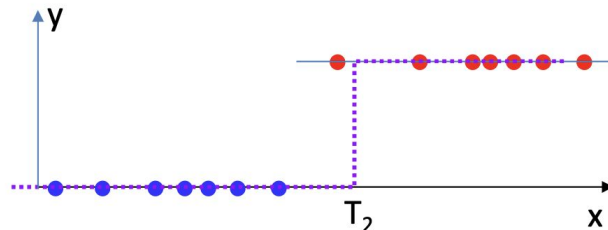
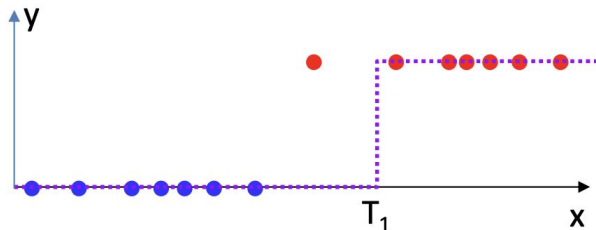
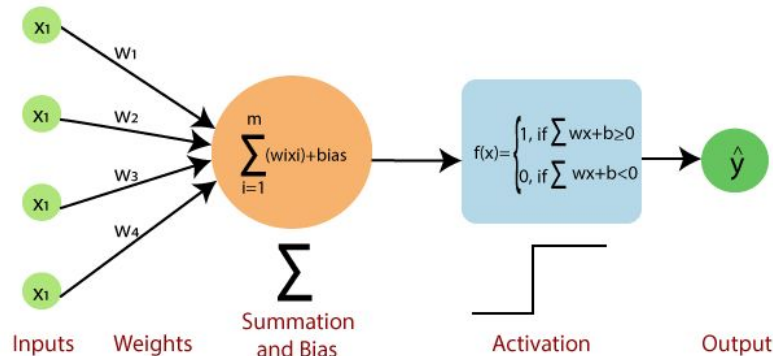
MLP as a continuous-valued regression



- A simple 3-unit MLP can generate a “square pulse” over an input
- **An MLP with many units can model an arbitrary function over an input**
 - To arbitrary precision
 - Simply make the individual pulses narrower
- **A one-hidden-layer MLP can model an arbitrary function of a single input**

Threshold Activations Are Not Enough

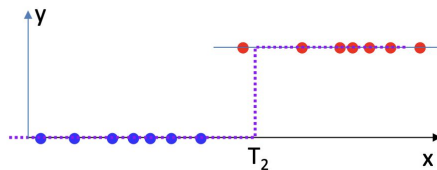
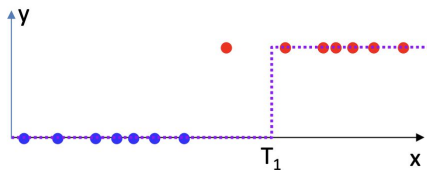
Threshold activation is step function:
discontinuous, no learning signals



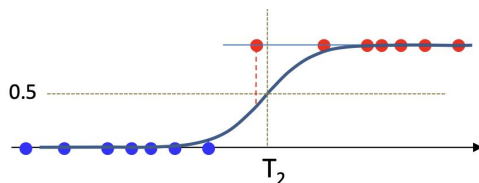
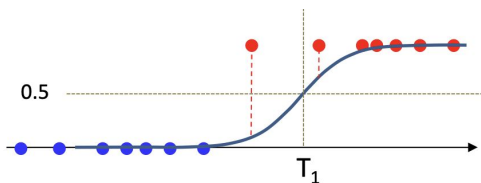
- Threshold activation: shifting the threshold from T_1 to T_2 does not change classification error
 - Does not indicate if moving the threshold left was good or not

Threshold Activations Are Not Enough

Differentiable Mismatch function

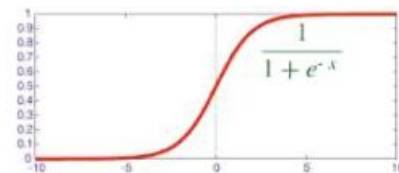


We want activation functions that are differentiable almost everywhere so gradients can guide learning.

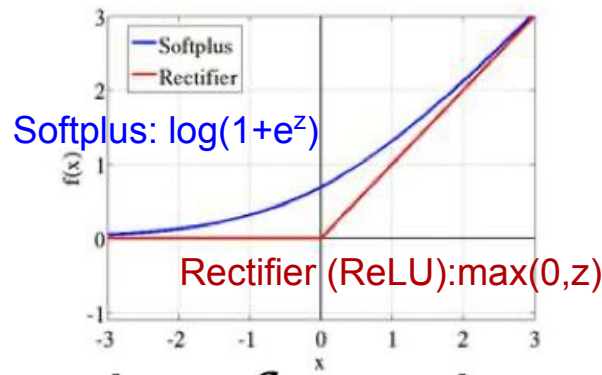


- Smooth, continuously varying activation: Classification based on whether the output is greater than 0.5 or less
 - Quantify *how much* the output differs from the desired target value (0 or 1)
 - Moving the function left or right changes this quantity, even if the classification error itself doesn't change

101



Sigmoid: $1/(1+e^{-x})$



Softplus: $\log(1+e^x)$

Rectifier (ReLU): $\max(0, x)$

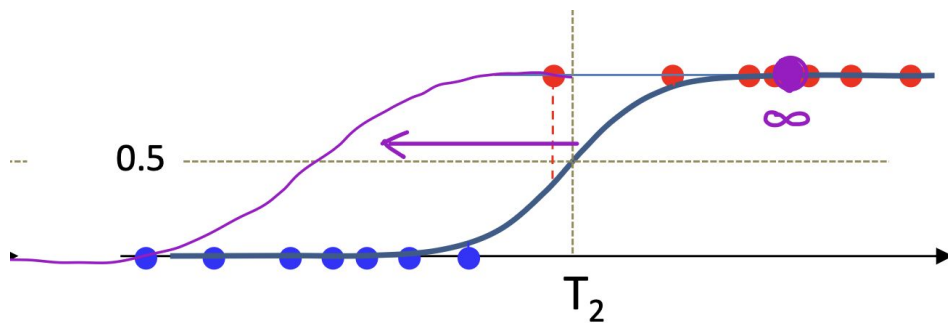
Mismatch (red dotted line)

= predicted output - target

= (activation output) - (true label)

Minimizing mismatch $\neq$ minimizing error

- When there are infinitely many **red** points, minimizing the total mismatch is dominated by that class
- The curve will shift left to reduce the aggregate mismatch of the **red** points, thus sacrifices a few **blue** points to better fit the dominant class
- As a result, a finite number of **blue** points may be misclassified



Hence, though we have a differentiable activation function, we can not measure classification accuracy using mismatch (activation output - true label)

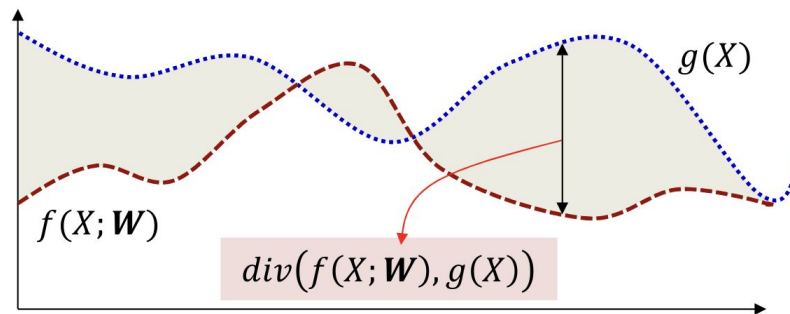
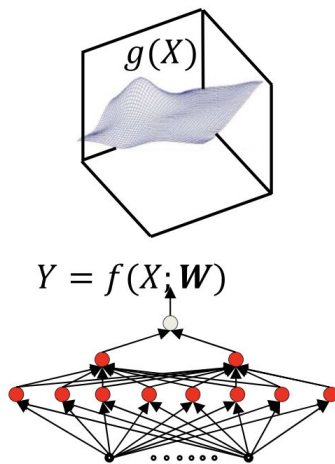
Then what is the right process of evaluating accuracy?

Definitions: g , f , div

$g(X)$ The true relationship between input and output (unknown in practice)

$f(X; \mathbf{W})$ The model, a function

$\text{div}(f(X; \mathbf{W}), g(X))$ Loss function, measure of discrepancy between model and truth

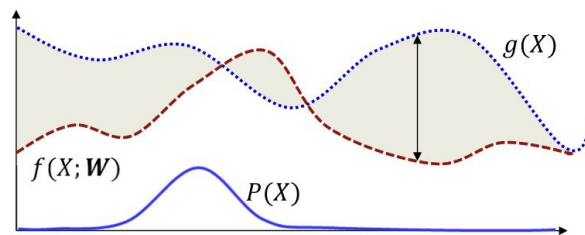
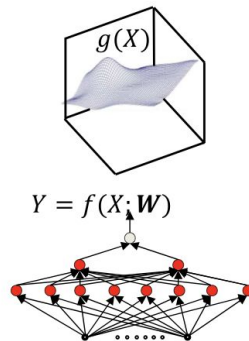


Expected divergence/loss (population level)

If X is a random variable
with probability density $P(X)$

$div(f(X; W), g(X))$ is pointwise loss,
here we want to integrate over
all X to get the overall
divergence

Minimizing *expected* divergence



$$\begin{aligned}\widehat{W} &= \operatorname{argmin}_W \int_X div(f(X; W), g(X)) P(X) dX \\ &= \operatorname{argmin}_W E[div(f(X; W), g(X))]\end{aligned}$$

Expected Prediction Error

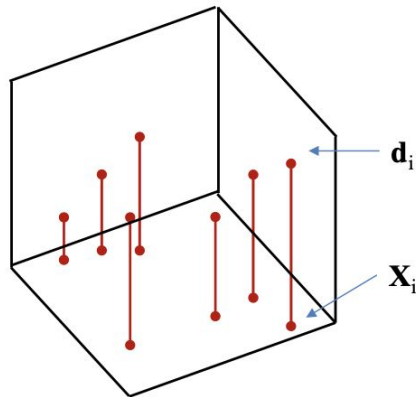
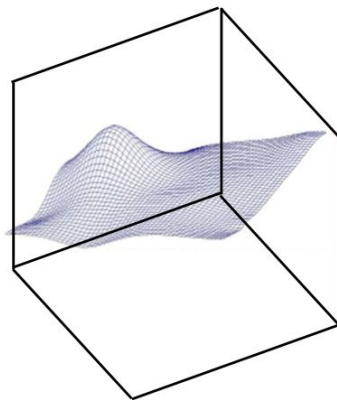
Empirical risk (sample level)

The *Empirical* risk

In practice, we only observe a finite set of data points.

Therefore, we approximate the expected loss using an empirical average over the observed samples.

This approximation is called the **empirical risk** (or loss).



The *expected* divergence (or risk) is the average divergence over the entire input space

$$E[\text{div}(f(X; W), g(X))] = \int_X \text{div}(f(X; W), g(X)) P(X) dX$$

The *empirical estimate* of the expected risk is the *average* divergence over the samples

$$E[\text{div}(f(X; W), g(X))] \approx \frac{1}{N} \sum_{i=1}^N \text{div}(f(X_i; W), d_i) \quad \text{Loss}(W)$$

Some loss functions

Loss functions for classification

- Binary cross-entropy (log loss)

$$\ell(y, \hat{y}) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$$

- Hinge loss

$$\ell(y, f(x)) = \max(0, 1 - yf(x))$$

Loss functions for regression

- Squared loss (L2 loss)

$$\ell(y, \hat{y}) = (y - \hat{y})^2$$

- Absolute loss (L1 loss)

$$\ell(y, \hat{y}) = |y - \hat{y}|$$

Reference

- NN structure (page 4&7): <https://lamarr-institute.org/blog/deep-neural-networks/>
- One-layer perceptron (page 4&8):
<https://medium.com/nerd-for-tech/flux-prediction-using-single-layer-perceptron-and-multilayer-perceptron-cf82c1341c33>
- (page 32): <https://builtin.com/machine-learning/common-loss-functions>
- All other figures from 11-785 course slides